StudyForge

AI Study and Learning Agent

Stage 1 Project Design Report

EECS3311 Software Design | Fall 2026

System type	Agent-based learning workspace
Interfaces	Graphical user interface and command-line interface
Primary model	OpenAI GPT-4.1 mini snapshot gpt-4.1-mini-2025-04-14
Design baseline	12 features, 12 use cases, 11 sequence diagrams, 7 patterns

Design date: October 9, 2026

Executive Summary

StudyForge is a dual-interface AI learning agent for university students who must turn fragmented course materials into reliable study actions. It ingests instructor-provided content, retrieves evidence, plans study work, creates practice, evaluates performance, and adapts future sessions. The design deliberately separates deterministic safeguards from model-driven generation so every generated artifact can be validated, traced to source content, and revised by the learner.

The system exposes the same application services through a graphical desktop interface and a command-line interface. Twelve non-trivial features are mapped to twelve use cases, eleven runtime sequence diagrams, named classes and methods, and seven course design patterns. The architecture treats the model as a replaceable infrastructure adapter rather than the application itself. This keeps domain rules, persistence, progress calculations, permissions, and session state testable without an AI service.

Contents

1 Project Overview
2 Detailed Feature Specifications
3 Overall Architecture
4 UML Class Diagram and Class Responsibilities
5 Design Pattern Explanations
6 Use-Case Model and Descriptions
7 Sequence Diagrams
8 Feature-to-Design Traceability
9 Feature Realization Explanations
10 Important Design Decisions and Quality Attributes
11 Stage 2 Implementation Boundary
References

1 Project Overview

1.1 Problem and Motivation

University students often store slides, readings, notes, and assessment results in separate places. Generic chatbots can summarize text, but they do not maintain course scope, source traceability, study-plan state, mastery evidence, or reliable multi-step workflows. The practical problem is not a lack of generated text; it is converting trusted course material and limited study time into a coherent learning process without losing control of evidence and progress.

1.2 Target Users

The primary users are university and college students managing one or more content-heavy courses. The design also supports accessibility-oriented workflows through keyboard-driven CLI access, predictable commands, resumable sessions, and exported plain-text results. The first release is single-user and local-first; instructor and administrator roles are outside Stage 1 scope.

1.3 Agent Purpose and Capabilities

StudyForge behaves as an agent because it maintains goals and context, decomposes requests into steps, selects tools, retrieves evidence, uses memory, checks intermediate results, changes its next action after feedback, and persists checkpoints. The agent may autonomously execute read-only and local operations inside a user-approved session. External side effects, such as calendar writes or replacing an active plan, require confirmation.

- Reasoning and planning: decomposes learning goals into feasible sessions and activity sequences.
- Retrieval: searches only indexed course chunks and returns stable source locations.
- Tool use: invokes ingestion, retrieval, reminder, repository, and activity tools through a registry.
- Memory: stores conversation turns, study plans, session checkpoints, attempts, and mastery events.
- Decision making: selects activity type and difficulty within deterministic pedagogical and safety constraints.
- Multi-step execution: runs stateful focused sessions with pause, resume, skip, retry, and stop transitions.

1.4 AI Models and Integration

The primary generation model is OpenAI GPT-4.1 mini using the pinned snapshot gpt-4.1-mini-2025-04-14 through the Responses API. The model is selected because official documentation identifies strong instruction following and tool calling, and documents support for function calling and structured outputs. Pinning the snapshot reduces unplanned behavior changes. ModelGateway isolates the application from the provider; OpenAIModelAdapter converts internal ModelRequest objects into API calls and validates ModelResult objects before they enter the domain.

Vector embeddings are generated behind a separate embedding port and stored in the local vector index. The design does not depend on a particular embedding vendor. All prompts receive only the retrieved chunks, current task, relevant learner state, and an output schema. The model cannot write repositories or external calendars directly; it can only propose typed actions that deterministic services validate and execute.

1.5 Scope

In scope	Out of scope for Stage 2 baseline
Course workspaces; local file ingestion; grounded	Learning-management-system integration; instructor

In scope	Out of scope for Stage 2 baseline
summaries and Q&A; concept maps; plans; flashcards; quizzes; grading; adaptation; analytics; reminders; focused sessions.	grading; social features; web-wide research; medical/psychological diagnosis; automatic submission of coursework; unsupervised external side effects.

1.6 Success Criteria

- Every major feature is available in both GUI and CLI through shared application services.
- Every generated factual artifact resolves to indexed evidence or is labeled as a suggestion.
- A learner can interrupt and resume an agent session without losing completed work.
- Deterministic calculations and policies are testable without network or model access.
- Feature, use-case, class, method, sequence, and pattern names remain traceable into Stage 2.

2 Detailed Feature Specifications

The twelve features below are meaningful application capabilities; none is a login, exit, settings-only, or informational screen. Each feature is reachable from both interfaces and uses the same controller and application services.

F01 Material Ingestion and Indexing

Description	Adds course files to an existing workspace and turns them into retrievable, traceable learning content.
GUI access	Course workspace > Add materials
CLI access	studyforge material import <paths> --course <id>
Input	Existing course ID and PDF, DOCX, TXT, or Markdown files.
Output	Validated materials, extracted text, chunks, metadata, and an indexing report.
AI involvement	Hybrid. AI proposes titles and topic tags; parsing, validation, hashing, chunking, embedding, and persistence are deterministic.
Expected workflow	Select an existing course, choose files, validate type and size, extract text, chunk content, generate local embeddings, infer metadata, persist the material, and publish MaterialIndexed.
Errors and alternatives	Reject unsupported/encrypted files, isolate per-file failures, detect duplicates by hash, and preserve successfully imported files.

F02 Grounded Material Summaries

Description	Produces concise or detailed summaries grounded in selected course content.
GUI access	Material viewer > Summarize
CLI access	studyforge summarize --material <id> --detail <level>

Input	Material or section ID, detail level, and optional focus question.
Output	Structured summary with key ideas, vocabulary, and source locations.
AI involvement	Hybrid. The model synthesizes retrieved passages; retrieval, citation resolution, and schema validation are deterministic.
Expected workflow	Retrieve relevant chunks, build a bounded prompt, generate structured output, verify every citation, save the summary, and present it.
Errors and alternatives	Return an evidence-gap message for weak retrieval; retry once for schema failure; never show unresolved citations.

F03 Concept Extraction and Knowledge Map

Description	Creates an evidence-linked graph of concepts and relationships.
GUI access	Course workspace > Concept map
CLI access	studyforge concepts build --course <id>
Input	Course ID and optional material/topic filters.
Output	Concept nodes, prerequisite/related-to edges, evidence links, and confidence scores.
AI involvement	Hybrid. AI proposes concepts and relationships; deterministic rules merge duplicates, enforce edge types, and attach evidence.
Expected workflow	Retrieve course chunks, extract candidate concepts, normalize names, merge duplicates, validate relationships, persist the graph, and render it.
Errors and alternatives	Quarantine unsupported relationships, flag low-confidence edges, and permit regeneration for selected materials.

F04 Grounded Question Answering

Description	Answers questions using only retrieved course evidence and shows where claims came from.
GUI access	Course chat > Ask
CLI access	studyforge ask --course <id> "<question>"
Input	Question, course scope, optional material filters, and conversation context.
Output	Answer with source citations, confidence label, and suggested follow-ups.
AI involvement	Hybrid. The agent plans retrieval, synthesizes evidence, and explains uncertainty; tools enforce scope and citation

	validity.
Expected workflow	Classify intent, create a retrieval plan, fetch and rank chunks, generate an answer, verify citations, store the turn, and display it.
Errors and alternatives	Clarify ambiguous questions, refuse unsupported claims, and report unavailable or insufficient sources.

F05 Personalized Study Plan

Description	Builds a feasible, personalized sequence of study sessions from goals and constraints.
GUI access	Planner > Create plan
CLI access	studyforge plan create --goal <text> --deadline <date>
Input	Learning goal, deadline, availability, priorities, and current mastery.
Output	Sequenced sessions with objectives, activities, durations, and checkpoints.
AI involvement	Hybrid. The agent decomposes goals and orders topics; deterministic constraints validate dates, total workload, and prerequisites.
Expected workflow	Load profile and mastery, derive constraints, plan topics and activities, validate feasibility, ask for approval, and persist the plan.
Errors and alternatives	Explain infeasible constraints, propose trade-offs, and require approval before replacing an existing plan.

F06 Flashcard Generation and Review

Description	Creates evidence-linked flashcards and schedules review from performance.
GUI access	Practice > Flashcards
CLI access	studyforge flashcards generate|review --topic <id>
Input	Topic/material scope, card count, difficulty, and review responses.
Output	Cited flashcard deck, review queue, correctness feedback, and updated intervals.
AI involvement	Hybrid. AI drafts cards and explanations; deterministic filters remove duplicates and spaced-repetition logic schedules reviews.
Expected workflow	Retrieve evidence, generate schema-constrained cards, validate and deduplicate, save the deck, run review, and record outcomes.
Errors and alternatives	Discard unsupported cards, cap regeneration, and allow

	users to edit or reject individual cards.

F07 Quiz Generation

Description	Creates validated quizzes aligned to selected topics and objectives.
GUI access	Practice > Generate quiz
CLI access	studyforge quiz generate --topic <id> --count <n>
Input	Topic scope, learning objectives, item count, formats, and difficulty.
Output	Quiz with objective/free-response items, answer key, rubrics, and citations.
AI involvement	Hybrid. AI drafts questions and rubrics; deterministic validators check schema, uniqueness, answerability, and requested coverage.
Expected workflow	Select objectives, retrieve evidence, generate candidate items, validate coverage and duplicates, save the quiz, and preview it.
Errors and alternatives	Remove invalid items, retry once when coverage is low, and label a partial quiz if the target count cannot be met.

F08 Automatic Grading and Explanations

Description	Scores attempts and explains answers while separating deterministic and model-based grading.
GUI access	Quiz attempt > Submit
CLI access	studyforge quiz grade --attempt <id>
Input	Quiz attempt, answer key/rubric, and learner answers.
Output	Per-item score, feedback, cited explanation, total score, and confidence.
AI involvement	Hybrid. Objective items use deterministic graders; AI evaluates free text against a rubric and drafts explanations.
Expected workflow	Validate completeness, grade objective items, evaluate free text, check confidence, calculate totals, save the attempt, and display feedback.
Errors and alternatives	Flag low-confidence grades for review, prevent double submission, and retain an auditable rubric/result record.

F09 Weak-Topic Diagnosis and Adaptive Practice

Description	Finds weak concepts and selects appropriately difficult next practice.
GUI access	Progress > Weak topics > Practice

CLI access	studyforge adapt --course <id> --duration <minutes>
Input	Attempt history, mastery estimates, study goal, time budget, and learner preferences.
Output	Ranked weak topics and a difficulty-adjusted next-practice recommendation.
AI involvement	Hybrid. AI explains patterns and chooses pedagogically suitable activity types; deterministic mastery thresholds and prerequisites constrain choices.
Expected workflow	Aggregate evidence, calculate mastery, identify weak prerequisites, select an adaptation strategy, generate practice, and request learner confirmation.
Errors and alternatives	Require minimum evidence, avoid overreacting to one attempt, and explain when no reliable diagnosis is possible.

F10 Progress and Mastery Analytics

Description	Shows reproducible learning metrics and optional AI explanations of trends.
GUI access	Dashboard > Progress
CLI access	studyforge progress show --course <id>
Input	Study events, attempts, plan completion, and date range.
Output	Mastery-by-topic, activity history, plan adherence, trends, and an optional narrative summary.
AI involvement	Hybrid. Metrics are deterministic; the model may explain trends without changing the underlying values.
Expected workflow	Project immutable events into progress snapshots, compute metrics, render charts/tables, and optionally generate a plain-language interpretation.
Errors and alternatives	Show data freshness and sample size, tolerate missing optional events, and never invent metrics for sparse data.

F11 Study Scheduling and Reminders

Description	Turns approved plan sessions into conflict-checked reminders and calendar events.
GUI access	Planner > Schedule
CLI access	studyforge reminder schedule --plan <id>
Input	Study plan, availability, timezone, reminder preferences, and calendar authorization.
Output	Conflict-checked schedule plus reminder/calendar event statuses.
AI involvement	Hybrid. AI suggests rescheduling options; deterministic

	validation handles timezones, overlaps, authorization, and idempotency.
Expected workflow	Read approved plan, find slots, detect conflicts, present options, obtain confirmation, write reminders, and record external IDs.
Errors and alternatives	Save pending reminders on service failure, retry idempotently, and never create external events without confirmation.

F12 Orchestrated Focused Study Session

Description	Runs a bounded multi-step study workflow that adapts after each activity.
GUI access	Dashboard > Start session
CLI access	studyforge session start --goal <text> --minutes <n>
Input	Session goal, duration, learner state, course scope, and preferred activity mix.
Output	A checkpointed sequence of explanation, recall, practice, feedback, and a session summary.
AI involvement	Hybrid. The agent plans and adapts multi-step work, chooses tools, and uses memory; deterministic state transitions, permissions, and time limits bound autonomy.
Expected workflow	Create a session plan, enter Setup, execute Learn/Practice/Review activities, adapt after each result, persist checkpoints, and complete with next actions.
Errors and alternatives	Pause safely on tool failure, allow skip/stop at checkpoints, preserve progress, and require confirmation for schedule changes.

3 Overall Architecture

The architecture follows dependency inversion: presentation depends on application services, domain logic depends on ports, and infrastructure adapters implement those ports. GUI and CLI never call the model or repositories directly. The same command created by CommandFactory is executed regardless of the boundary that initiated it.

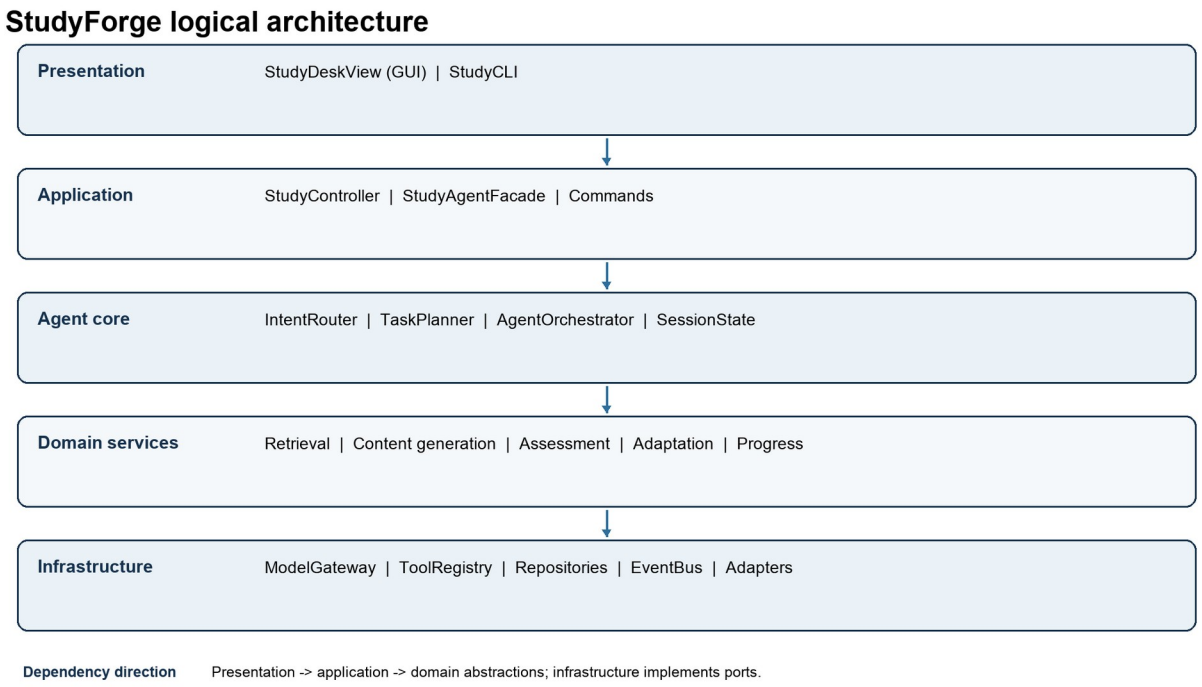


Figure 1. Logical architecture and dependency direction.

3.1 Component Responsibilities

Layer	Main components	Responsibility
Presentation	StudyDeskView, StudyCLI	Collect input, render results, and translate UI events or CLI arguments into requests.
Application	StudyController, StudyCommand, CommandFactory, StudyAgentFacade	Coordinate use cases, transaction boundaries, confirmations, and undoable local actions.
Agent core	IntentRouter, TaskPlanner, AgentOrchestrator, SessionState	Classify goals, plan steps, select tools, advance state, and recover from failures.
Domain	Course, Material, StudyPlan, StudySession, Quiz, Attempt, ProgressSnapshot	Protect invariants and represent learning state independent of frameworks.
Services	RetrievalService, AssessmentService, AdaptationService, ProgressService	Implement cohesive business operations and deterministic safeguards.
Infrastructure	OpenAIModelAdapter, tools, repositories, EventBus, external adapters	Connect model, storage, local files, vector index, and calendar behind interfaces.

4 UML Class Diagram and Class Responsibilities

UML class diagram

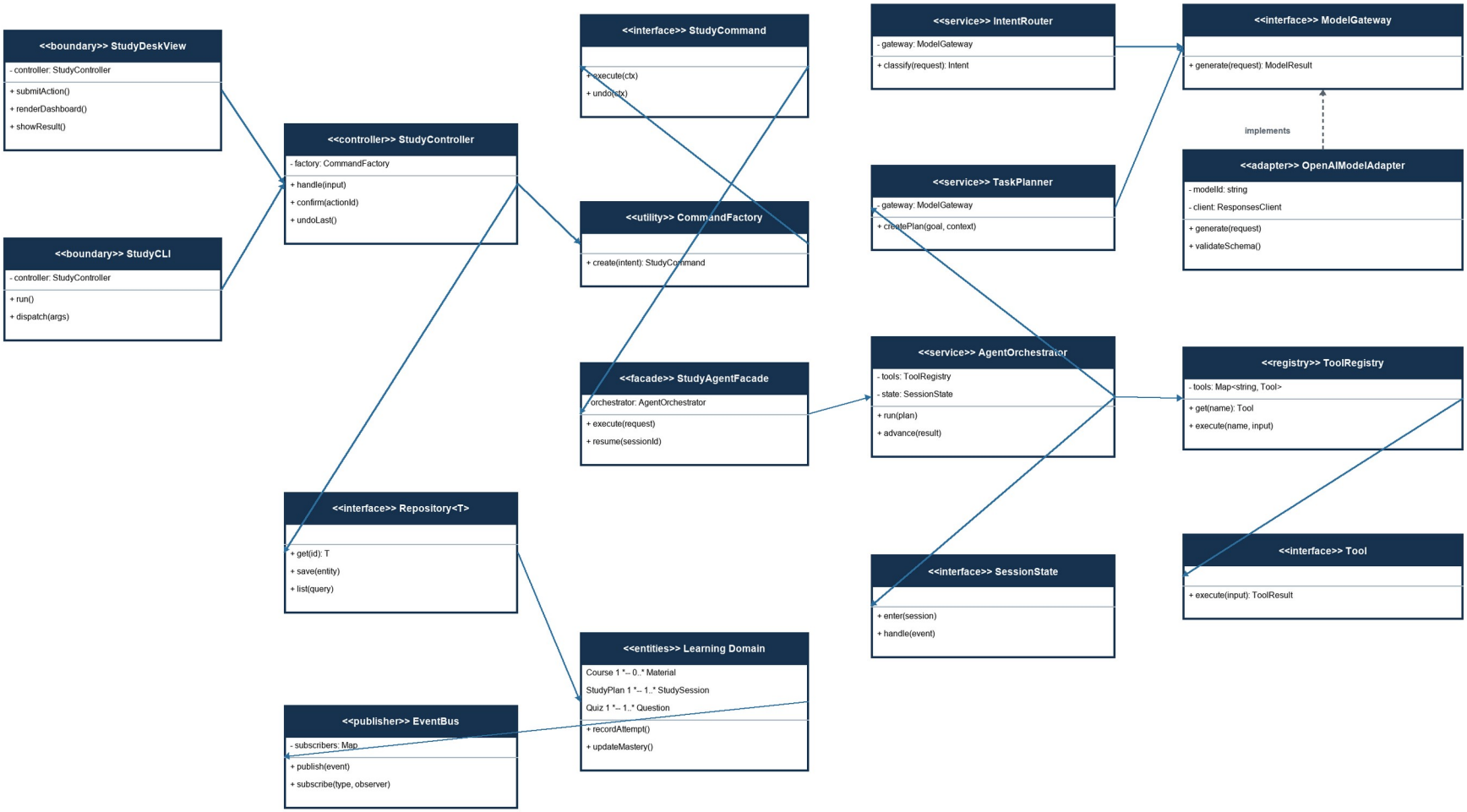


Figure 2A. Core application and agent classes. Solid arrows denote dependencies and dashed arrows denote interface realization.

4 UML Class Diagram Continued

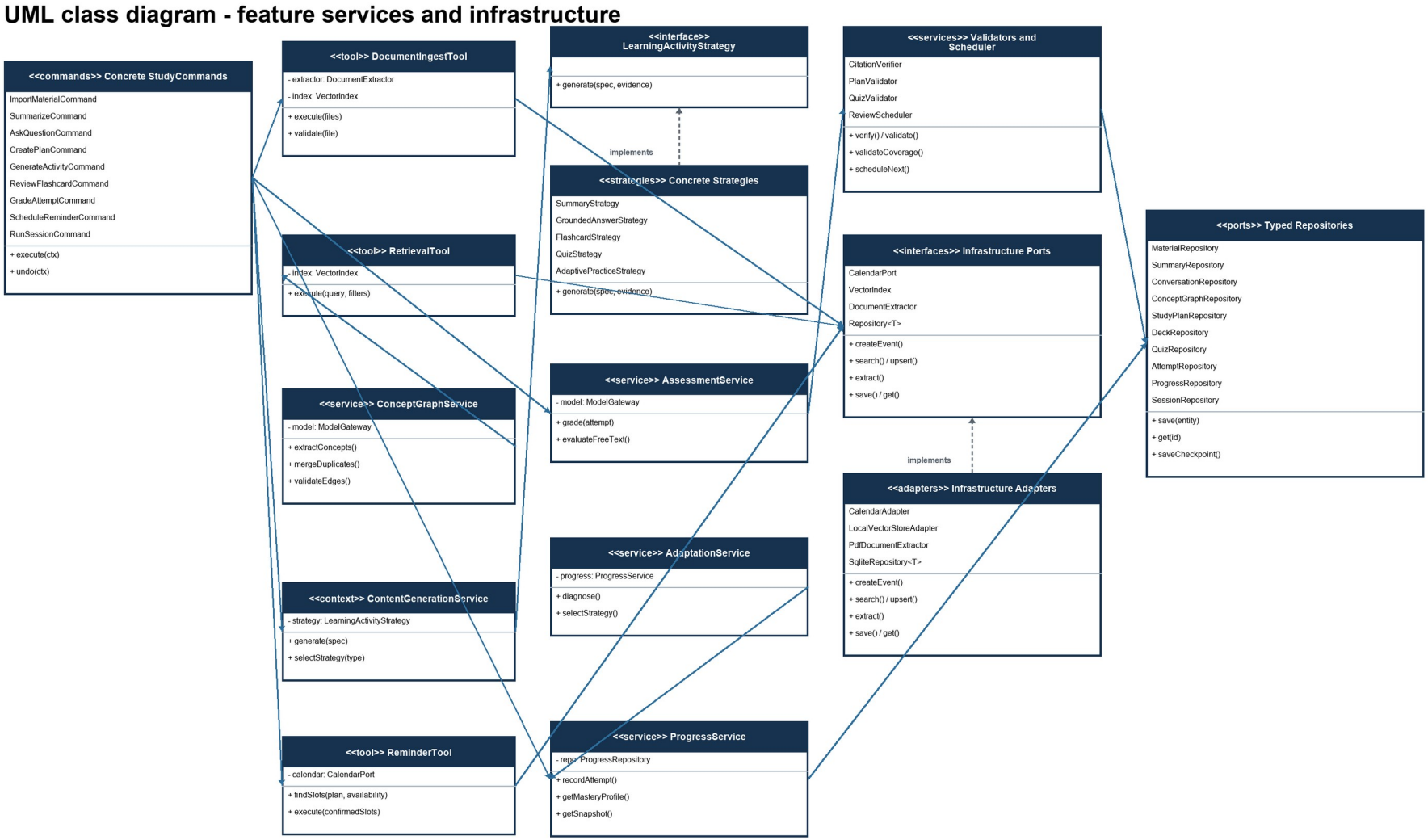


Figure 2B. Feature services, Strategy participants, validators, adapters, and repositories used by the feature mappings and sequence diagrams.

4 UML Class Diagram Continued

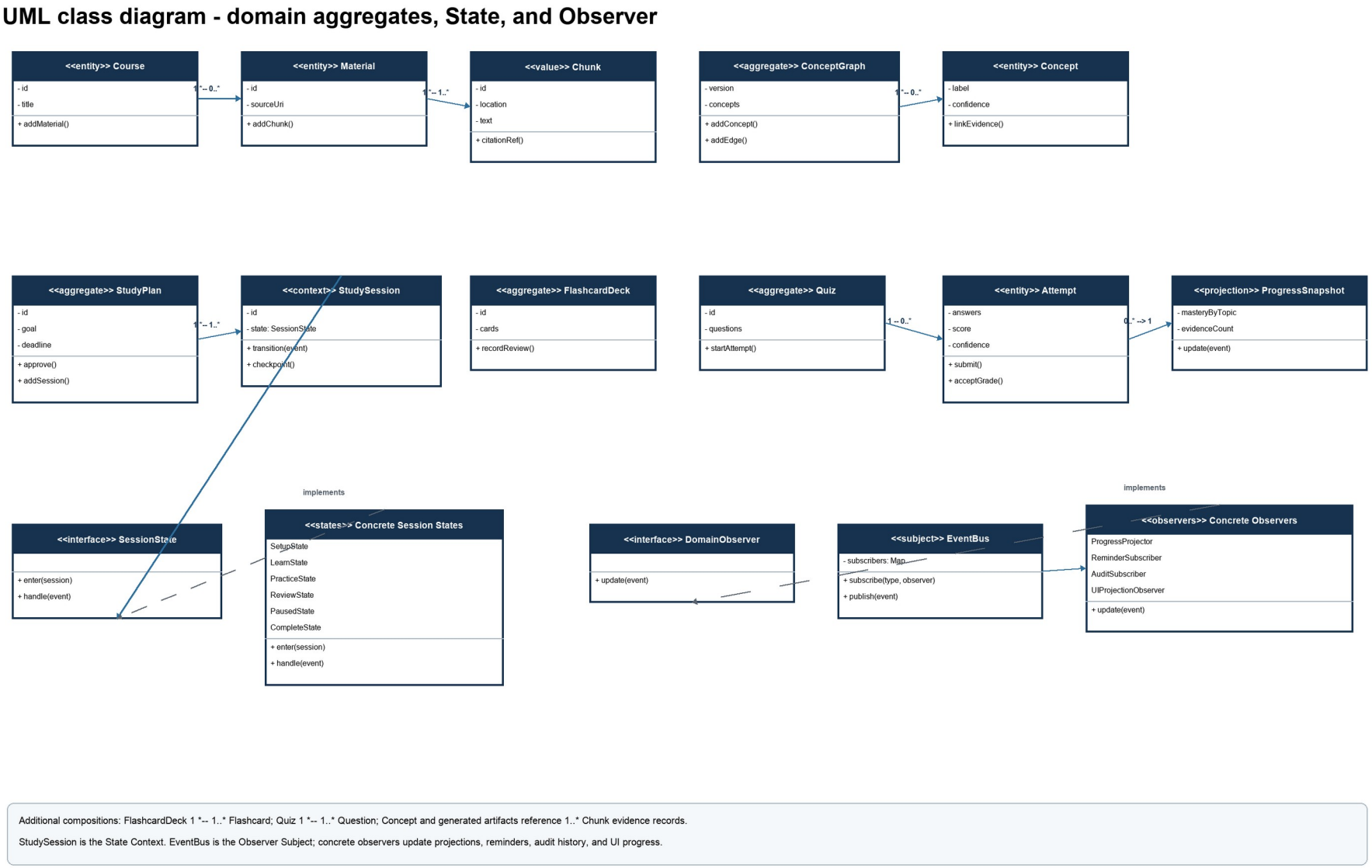


Figure 2C. Domain aggregates with composition and multiplicities, plus explicit State and Observer roles and realizations.

4.1 Class and Interface Catalog

Class/interface	Role	Responsibility
StudyDeskView / StudyCLI	Boundary	Render the same use cases for graphical and command-line interaction.
StudyController	Controller	Validate boundary requests, create commands, and manage confirmation/undo flow.
StudyCommand	Interface	Represent an executable use-case request; concrete commands encapsulate orchestration.
CommandFactory	Utility	Map a validated Intent to a concrete StudyCommand; it is not counted as a design pattern.
StudyAgentFacade	Facade	Offer one application entry point for agent execution and session resumption.
IntentRouter	Service	Classify request intent and required scope.
TaskPlanner	Service	Create typed PlanStep sequences under constraints.
AgentOrchestrator	Service	Execute steps, call tools, persist checkpoints, and choose recovery paths.
SessionState	Interface	Encapsulate legal behavior for Setup, Learn, Practice, Review, Paused, and Complete states.
Feature services	Services	Ingest, retrieve, build concepts, generate activities, grade, adapt, calculate progress, validate, and schedule review.
LearningActivityStrategy	Interface	Generate a typed learning artifact from a specification and retrieved evidence.
ModelGateway	Port	Expose provider-neutral structured generation.
OpenAIModelAdapter	Adapter	Translate ModelRequest/ModelResult to and from the OpenAI Responses API.
Tool / ToolRegistry	Port and registry	Provide typed tool execution and lookup without condition-heavy orchestration.
Repository<T>	Port	Persist and query aggregate roots through typed repositories.
EventBus / DomainObserver	Observer roles	Publish domain events to progress, reminder, audit, and UI observers.
Domain aggregates	Entities	Protect course, material, concept graph, plan, session, deck, quiz, attempt, and mastery invariants.

4.2 Relationship Detail

- StudyDeskView and StudyCLI each depend on exactly one StudyController; neither owns domain logic.
- StudyController depends on one CommandFactory and invokes zero or more StudyCommand instances over its lifetime.
- StudyAgentFacade composes one AgentOrchestrator; the orchestrator aggregates a ToolRegistry and a current SessionState.
- OpenAIModelAdapter realizes ModelGateway; concrete Tool implementations realize Tool; concrete repositories realize Repository<T>.
- Course composes zero or more Material objects. StudyPlan composes one or more StudySession objects. Quiz composes one or more Question objects. Their children do not exist independently of the aggregate.
- EventBus aggregates zero or more observers per event type. ProgressProjector and ReminderSubscriber observe domain events without coupling producers to consumers.

5 Design Pattern Explanations

P01 Model View Controller

Problem: GUI and CLI must share behavior without duplicating domain logic.

Participants: StudyDeskView and StudyCLI are views/boundaries; StudyController interprets actions; domain entities and repositories are the model.

Why appropriate: Both interfaces invoke identical use cases while remaining independently replaceable and testable.

Without it: Interface code would absorb workflow rules, drift between GUI and CLI, and become hard to test.

P02 Facade

Problem: Boundary code should not coordinate routers, planners, tools, states, and model calls.

Participants: StudyAgentFacade is the facade; AgentOrchestrator, TaskPlanner, ToolRegistry, ModelGateway, and repositories are subsystem classes.

Why appropriate: A small execute/resume API preserves separation of concerns and provides a stable boundary for commands.

Without it: Every command would know subsystem ordering and error handling, increasing coupling and inconsistency.

P03 Strategy

Problem: Summaries, grounded answers, flashcards, quizzes, and adaptive activities use different generation and validation rules.

Participants: LearningActivityStrategy defines generate(spec); SummaryStrategy, GroundedAnswerStrategy, FlashcardStrategy, QuizStrategy, and AdaptivePracticeStrategy implement it; ContentGenerationService selects a strategy.

Why appropriate: Algorithms vary independently and can be tested, replaced, or extended without branching across the service.

Without it: A large conditional method would mix prompts, validators, and persistence rules for unrelated activity types.

P04 Command

Problem: GUI clicks and CLI commands must become uniform, loggable, confirmable, and sometimes undoable operations.

Participants: StudyCommand is the Command; concrete commands encapsulate requests; StudyController is the Invoker; feature services are Receivers; RequestContext is command state.

Why appropriate: The same operation can be queued, audited, retried, or undone regardless of its presentation boundary.

Without it: Each boundary would implement its own orchestration and cross-cutting behavior.

P05 Adapter

Problem: External model, calendar, vector, and file APIs have unstable provider-specific shapes.

Participants: ModelGateway, CalendarPort, VectorIndex, and DocumentExtractor are Targets; OpenAIModelAdapter, CalendarAdapter, LocalVectorStoreAdapter, and format extractors are Adapters.

Why appropriate: Domain services use stable ports, tests use fakes, and providers can change without rewriting use cases.

Without it: Provider DTOs and exceptions would leak through the application and make offline testing difficult.

P06 Observer

Problem: Attempts, imports, and sessions must update progress, reminders, and audit history without tightly coupling producers.

Participants: EventBus is the Subject/publisher; DomainObserver is the Observer interface; ProgressProjector, ReminderSubscriber, AuditSubscriber, and UIProjectionObserver are concrete Observers.

Why appropriate: New projections and notifications can be added without changing the feature that emits the event.

Without it: Every service would call every downstream consumer directly, creating cyclic and fragile dependencies.

P07 State

Problem: A focused session permits different actions and transitions during Setup, Learn, Practice, Review, Paused, and Complete.

Participants: SessionState is the State interface; concrete state classes enforce legal transitions; StudySession is the Context; AgentOrchestrator delegates state-specific behavior.

Why appropriate: Transition rules are explicit, recoverable, and testable instead of being scattered booleans.

Without it: Combinations of flags would allow illegal actions and make pause/resume recovery unreliable.

6 Use-Case Model and Descriptions

UML use-case diagram

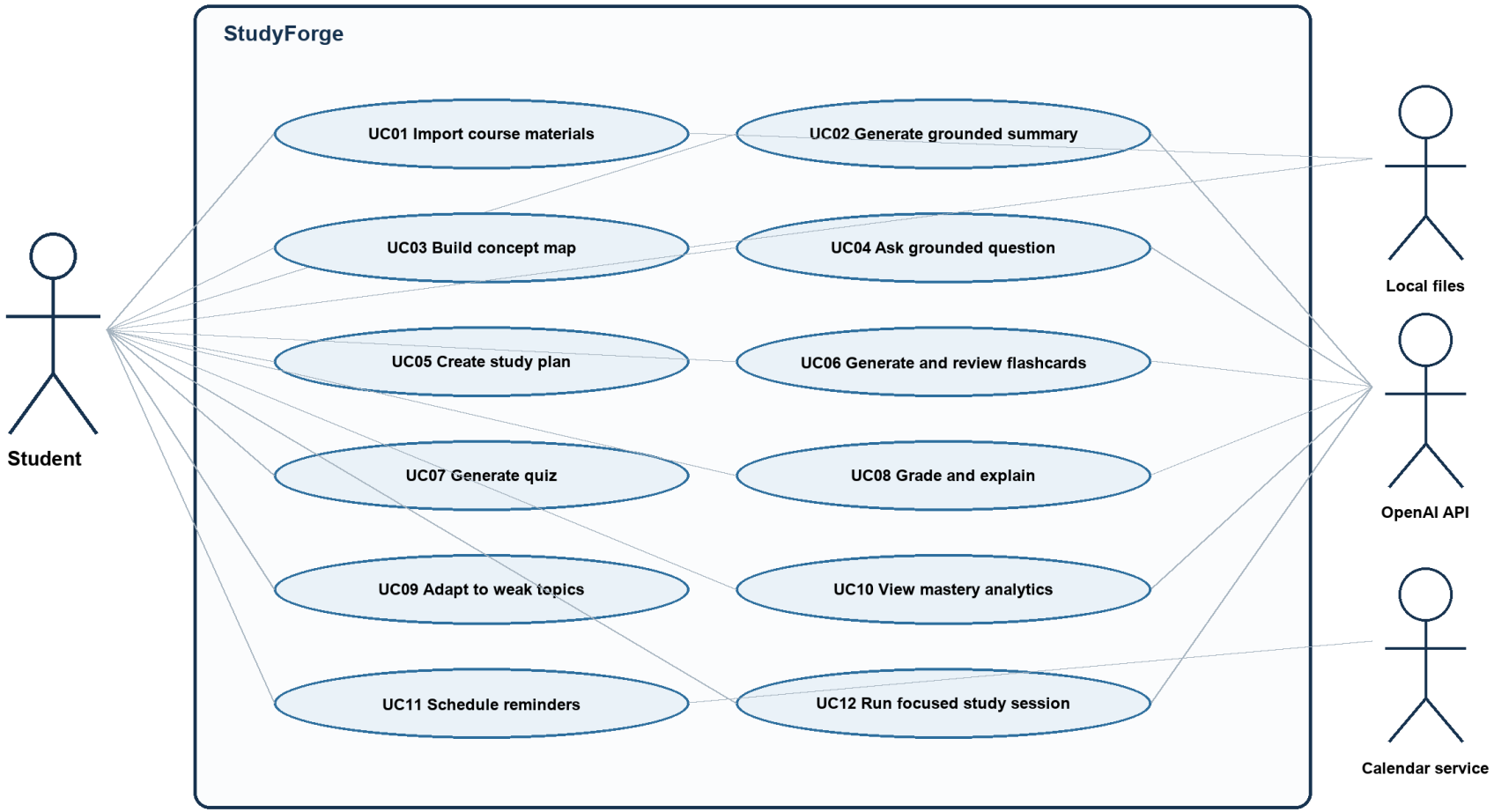


Figure 3. Student-facing use cases and external supporting actors. GUI and CLI are alternate boundaries for every use case.

UC01 Import Course Materials

Actors	Student; local file system; OpenAI API
Goal	Add trusted course content to an indexed workspace.
Preconditions	A course exists; file access is available.
Trigger	The student chooses Add materials or runs the import command.
Main success scenario	1. Select a course and one or more files. 2. Validate file types, sizes, and duplicates. 3. Extract and chunk text. 4. Generate local embeddings and proposed metadata. 5. Persist materials and publish MaterialIndexed. 6. Show the per-file indexing report.
Alternative and exception flows	Unsupported or encrypted files are rejected individually; duplicate files are linked or skipped; transient extraction failures can be retried.
Postconditions	Valid materials are stored, indexed, and available for retrieval.
Related features	F01

UC02 Generate Grounded Summary

Actors	Student; OpenAI API
Goal	Understand selected content quickly without losing traceability.
Preconditions	At least one indexed material exists.
Trigger	The student requests a summary and detail level.
Main success scenario	1. Resolve the selected scope. 2. Retrieve relevant chunks. 3. Build a schema-constrained prompt. 4. Generate and parse a summary. 5. Verify citations against chunk locations. 6. Save and display the result.
Alternative and exception flows	If evidence is weak, return an evidence-gap response; if schema validation fails, retry once; unresolved citations block publication.
Postconditions	A cited summary is saved for the selected scope.
Related features	F02

UC03 Build Concept Map

Actors	Student; OpenAI API
Goal	See key concepts and their supported relationships.
Preconditions	Course materials are indexed.
Trigger	The student opens Concept map or runs the build command.

Main success scenario	1. Retrieve chunks across the selected scope. 2. Extract concept and relationship candidates. 3. Normalize labels and merge duplicates. 4. Validate edge types and evidence. 5. Persist the graph. 6. Render nodes, edges, confidence, and sources.
Alternative and exception flows	Low-confidence edges are flagged; unsupported edge types are quarantined; selected sources can be regenerated.
Postconditions	A versioned, evidence-linked concept graph exists.
Related features	F03

UC04 Ask a Grounded Question

Actors	Student; OpenAI API
Goal	Receive an evidence-based answer from course materials.
Preconditions	The course contains indexed material.
Trigger	The student submits a question.
Main success scenario	1. Classify the question and resolve scope. 2. Plan retrieval. 3. Retrieve and rank evidence. 4. Generate a structured answer. 5. Verify citations and confidence. 6. Store the conversation turn and present follow-ups.
Alternative and exception flows	Ambiguous scope triggers clarification; insufficient evidence produces an explicit limitation; service failure preserves the question for retry.
Postconditions	The answer and its evidence trail are stored in conversation memory.
Related features	F04

UC05 Create Personalized Study Plan

Actors	Student; OpenAI API
Goal	Convert a goal and constraints into a feasible plan.
Preconditions	A learner profile and course exist.
Trigger	The student enters a goal, deadline, and availability.
Main success scenario	1. Load goals, preferences, mastery, and prerequisites. 2. Derive hard and soft constraints. 3. Generate a candidate plan. 4. Validate workload, dates, and ordering. 5. Present trade-offs for approval. 6. Persist the approved plan.
Alternative and exception flows	Infeasible constraints produce alternatives; replacement of an active plan requires confirmation; missing mastery uses a diagnostic baseline.
Postconditions	An approved, versioned StudyPlan exists.

Related features	F05

UC06 Generate and Review Flashcards

Actors	Student; OpenAI API
Goal	Practice recall with evidence-linked cards.
Preconditions	Relevant indexed content exists.
Trigger	The student requests or opens a deck.
Main success scenario	1. Resolve topic, count, and difficulty. 2. Retrieve supporting chunks. 3. Generate and validate cards. 4. Deduplicate and save the deck. 5. Present cards in a review session. 6. Record results and schedule the next review.
Alternative and exception flows	Unsupported or duplicate cards are removed; the student may edit/reject cards; a partial deck is labeled when regeneration is exhausted.
Postconditions	A deck and updated review schedule are stored.
Related features	F06

UC07 Generate Quiz

Actors	Student; OpenAI API
Goal	Create a balanced assessment for selected objectives.
Preconditions	Indexed content and objectives are available.
Trigger	The student specifies scope, formats, and difficulty.
Main success scenario	1. Select objectives and coverage targets. 2. Retrieve evidence. 3. Generate candidate items and rubrics. 4. Validate answerability, uniqueness, and coverage. 5. Persist the quiz. 6. Show a preview before starting.
Alternative and exception flows	Invalid items are removed; one bounded retry fills gaps; an incomplete target count is clearly reported.
Postconditions	A validated quiz and answer key are stored.
Related features	F07

UC08 Grade Attempt and Explain

Actors	Student; OpenAI API
Goal	Receive fair scoring and actionable feedback.
Preconditions	A quiz attempt exists and is not finalized.
Trigger	The student submits answers.

Main success scenario	1. Validate the attempt. 2. Grade objective items deterministically. 3. Evaluate free text against rubrics. 4. Calculate totals and confidence. 5. Persist the auditable result. 6. Display cited explanations.
Alternative and exception flows	Low-confidence free-text grades await review; duplicate submission returns the existing result; invalid attempts are rejected.
Postconditions	The attempt is finalized and a mastery event is emitted.
Related features	F08

UC09 Adapt Practice to Weak Topics

Actors	Student; OpenAI API
Goal	Choose the next useful practice based on evidence.
Preconditions	Sufficient attempt or diagnostic evidence exists.
Trigger	The student chooses Practice weak topics.
Main success scenario	1. Aggregate recent evidence. 2. Calculate mastery and uncertainty. 3. Find weak concepts and prerequisites. 4. Select an activity and difficulty strategy. 5. Generate the practice set. 6. Explain the recommendation and request confirmation.
Alternative and exception flows	Sparse evidence triggers a diagnostic rather than a diagnosis; one anomalous attempt cannot dominate; the learner may override the recommendation.
Postconditions	A recommended or accepted adaptive activity is recorded.
Related features	F09

UC10 View Progress and Mastery

Actors	Student; OpenAI API for optional narrative
Goal	Inspect learning progress using reproducible metrics.
Preconditions	Study or assessment events exist.
Trigger	The student opens Progress or runs the progress command.
Main success scenario	1. Load the requested date range. 2. Project events into mastery snapshots. 3. Calculate plan adherence and activity trends. 4. Render metrics and evidence counts. 5. Optionally generate a narrative explanation.
Alternative and exception flows	Missing optional events are ignored with a freshness notice; sparse data suppresses unreliable trends; calculation errors are logged and shown safely.
Postconditions	The dashboard reflects the latest successfully processed events.

Related features	F10

UC11 Schedule Study and Reminders

Actors	Student; calendar service
Goal	Place approved study sessions on a calendar without conflicts.
Preconditions	An approved plan exists; timezone is known.
Trigger	The student chooses Schedule or runs the reminder command.
Main success scenario	1. Read plan and availability. 2. Find candidate slots and conflicts. 3. Present choices. 4. Obtain confirmation. 5. Create reminders idempotently. 6. Store external IDs and show status.
Alternative and exception flows	Authorization failure leaves reminders pending; service failure can be retried; conflicting events produce alternative slots.
Postconditions	Confirmed reminders are stored and linked to plan sessions.
Related features	F11

UC12 Run Focused Study Session

Actors	Student; OpenAI API
Goal	Complete a bounded, adaptive learning workflow.
Preconditions	A course exists and session duration is positive.
Trigger	The student starts a focused session.
Main success scenario	1. Create a time-bounded activity plan. 2. Enter Setup and confirm scope. 3. Execute Learn and Practice activities through tools. 4. Evaluate each result and adapt the next step. 5. Checkpoint after every transition. 6. Enter Review, summarize progress, and suggest next actions.
Alternative and exception flows	The student may pause, skip, or stop; tool failure enters a recoverable state; external side effects require confirmation.
Postconditions	The session, results, and next actions are persisted.
Related features	F12

7 Sequence Diagrams

Eleven sequence diagrams cover every distinct runtime interaction shape. The traceability table maps all twelve features to at least one diagram. Messages use class and method names from the class model; each diagram includes a principal failure or alternative path.

SD01 Ingest and index materials

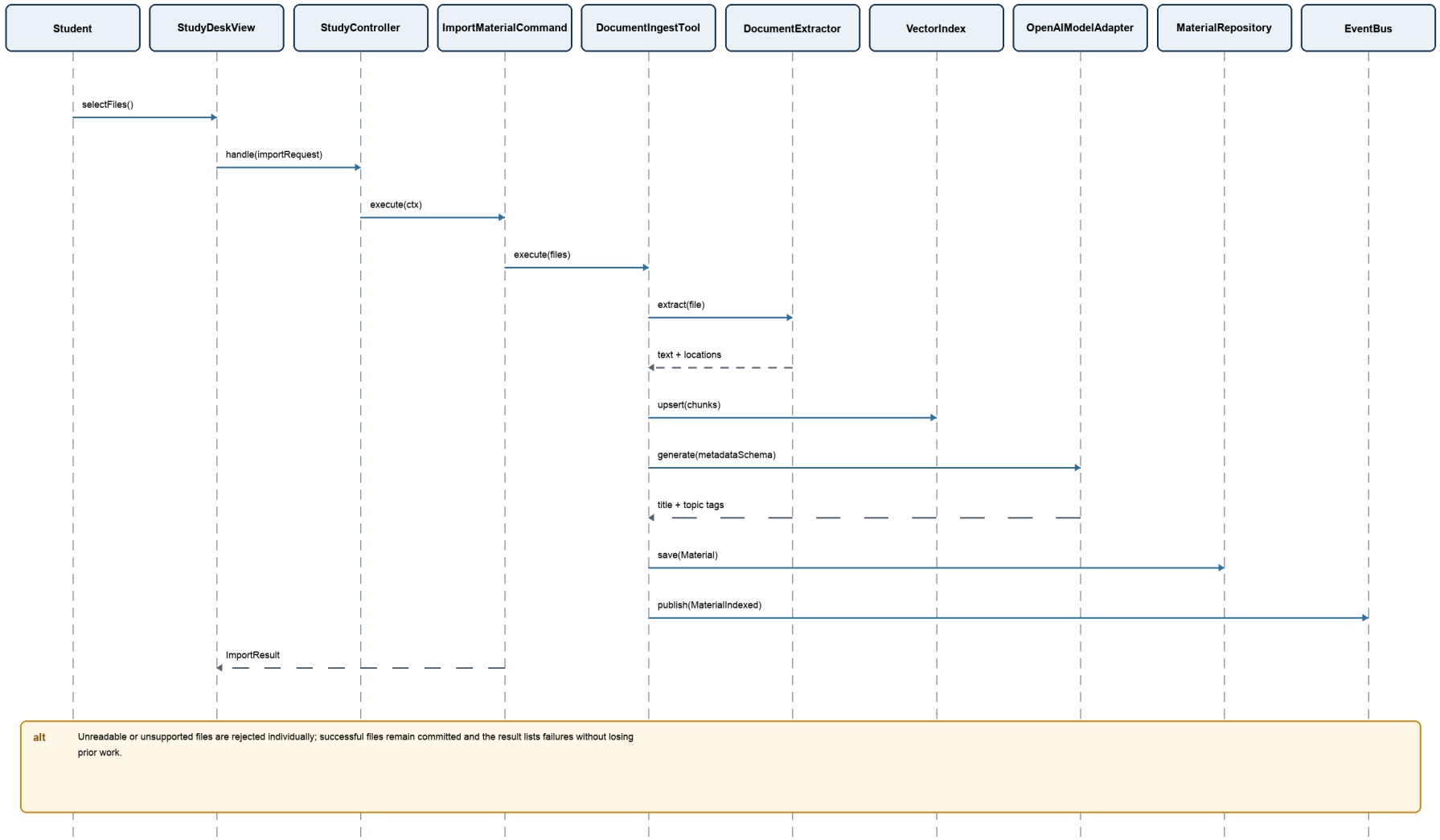


Figure 4. Ingest and index materials. Feature coverage: F01.

SD02 Grounded summary and question answering

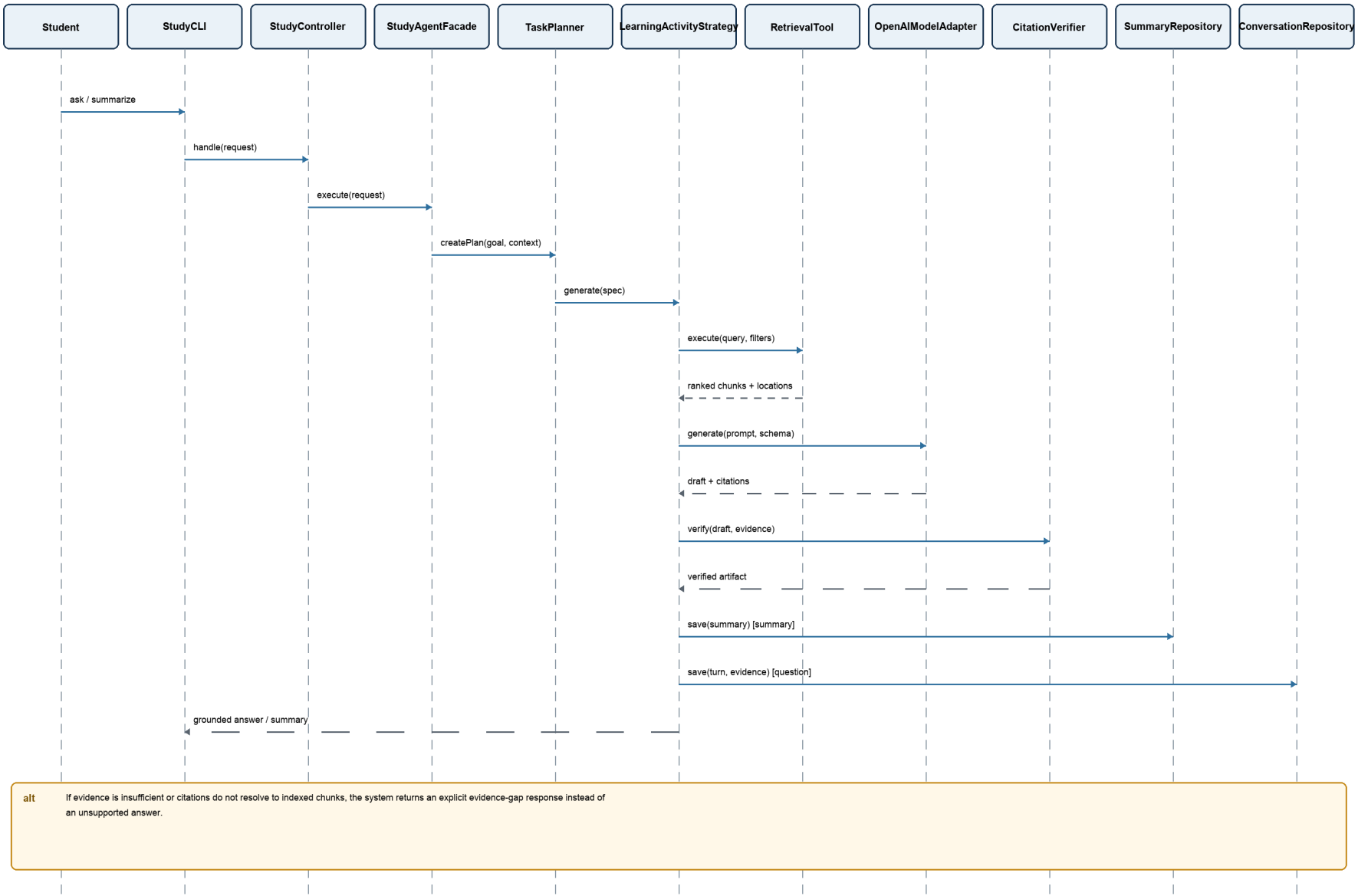


Figure 5. Grounded summary and question answering. Feature coverage: F02, F04.

SD03 Create and approve a study plan

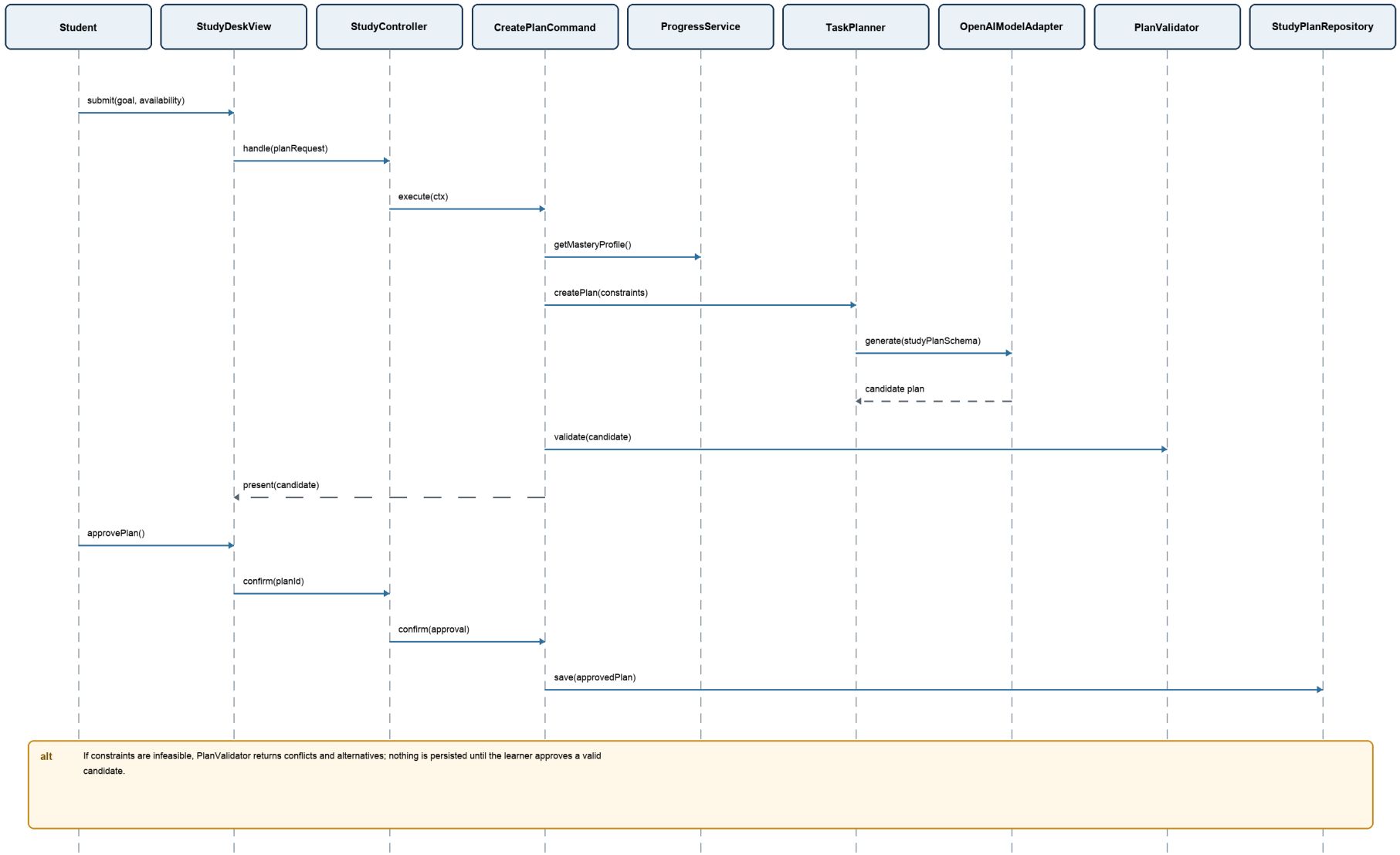


Figure 6. Create and approve a study plan. Feature coverage: F05.

SD04 Generate flashcards and quizzes

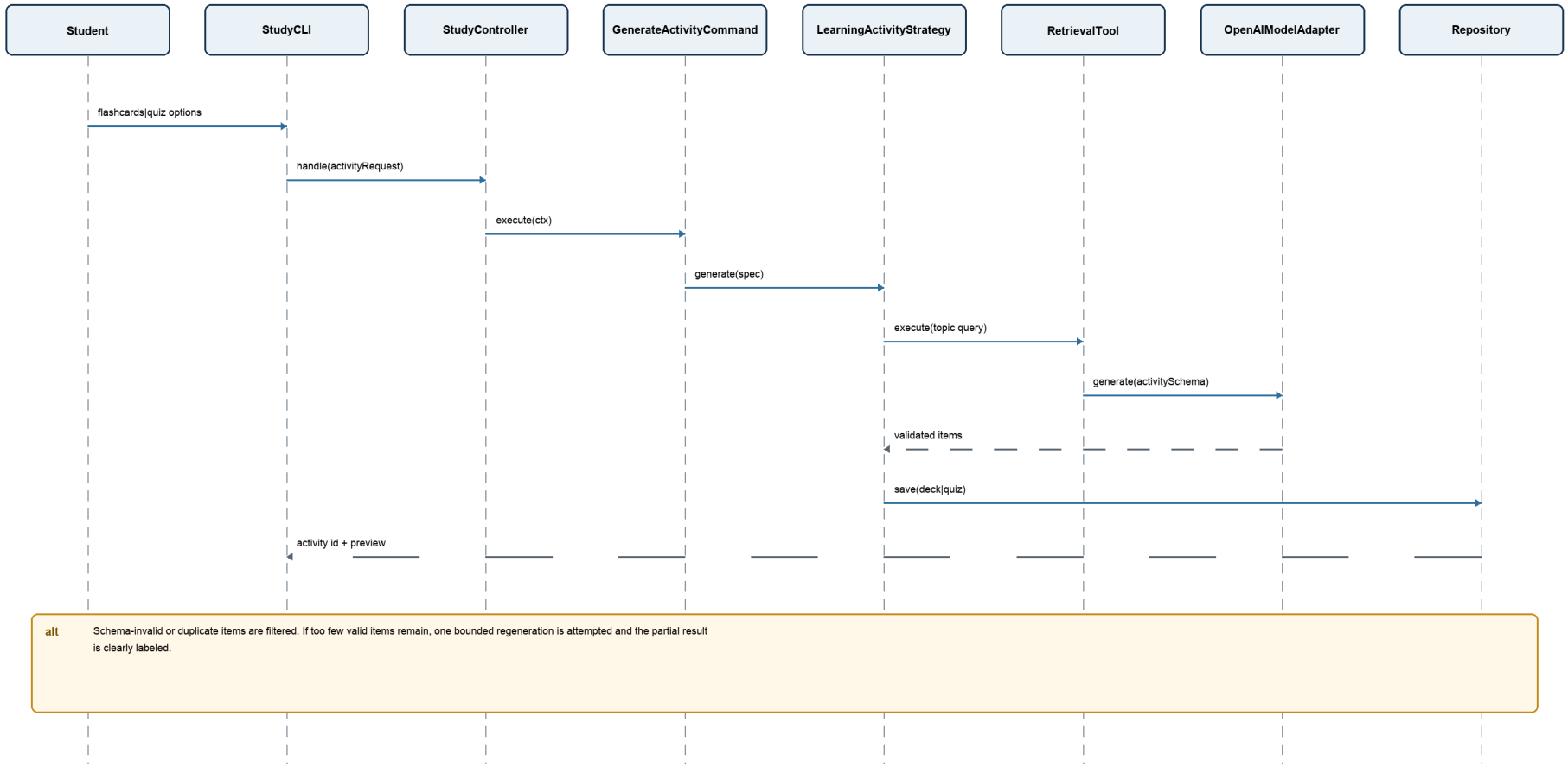


Figure 7. Generate flashcards and quizzes. Feature coverage: F06, F07.

SD05 Grade an attempt and explain results

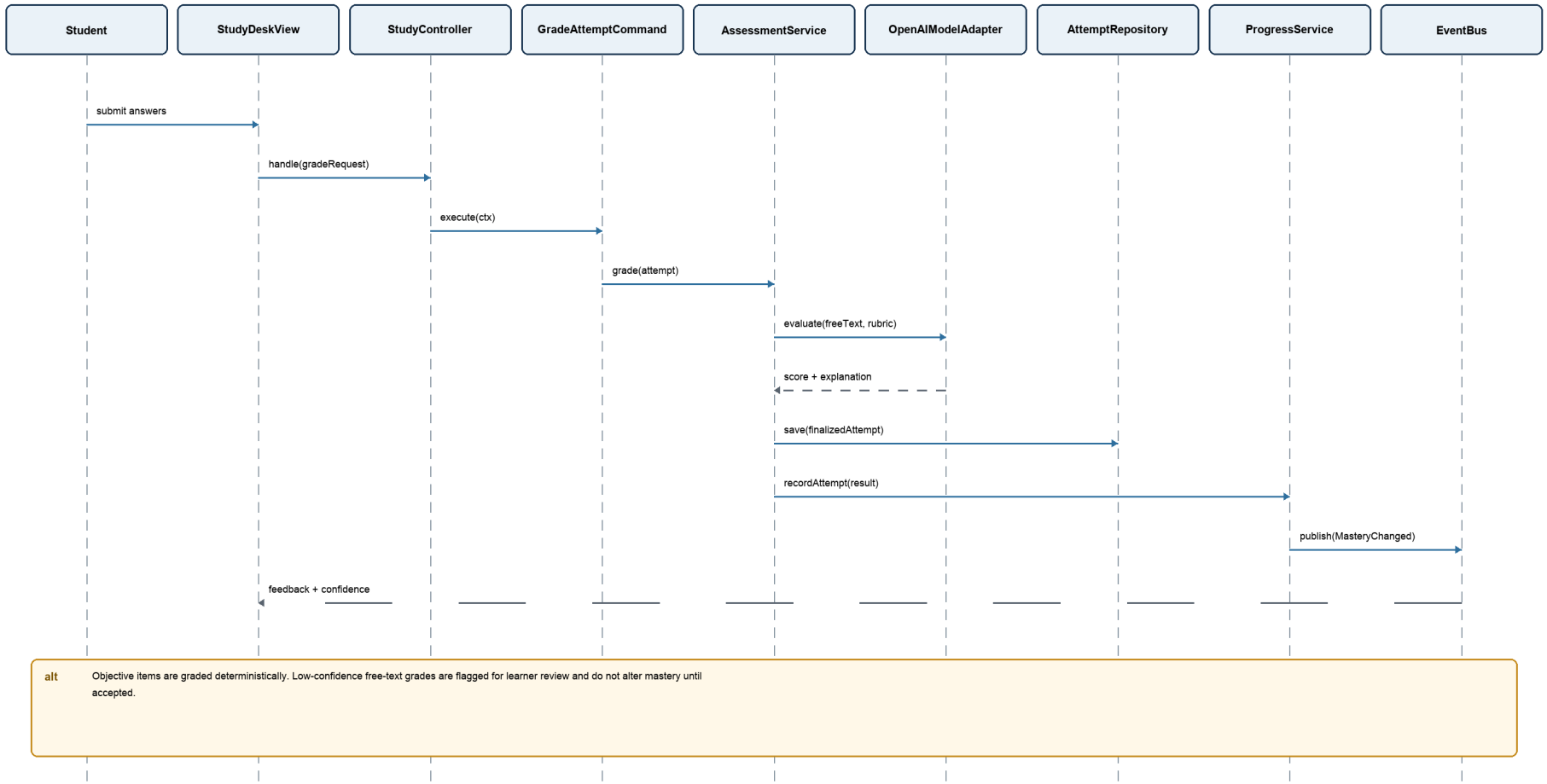


Figure 8. Grade an attempt and explain results. Feature coverage: F08.

SD06 Run an orchestrated study session

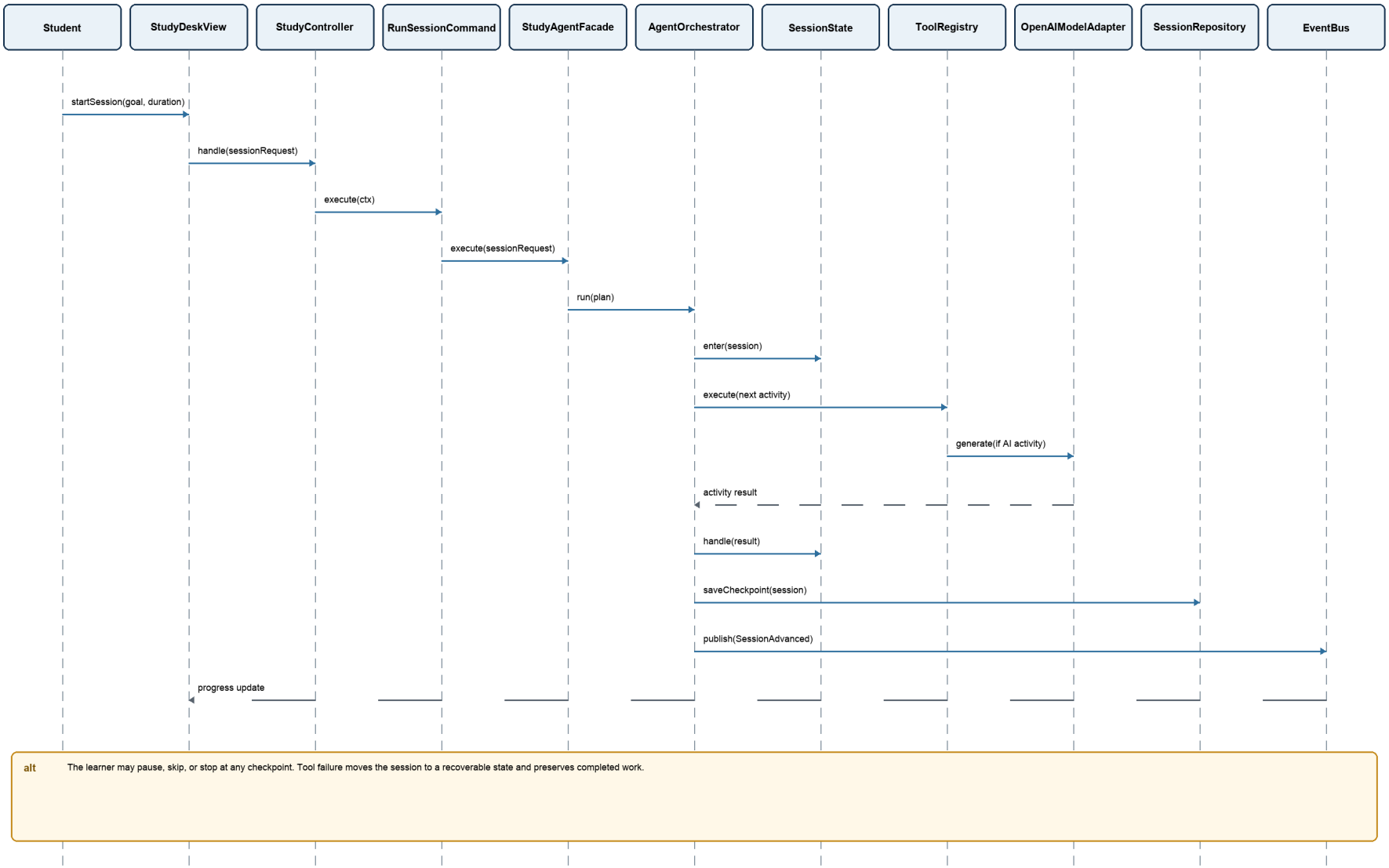


Figure 9. Run an orchestrated study session. Feature coverage: F12.

SD07 Build an evidence-linked concept map

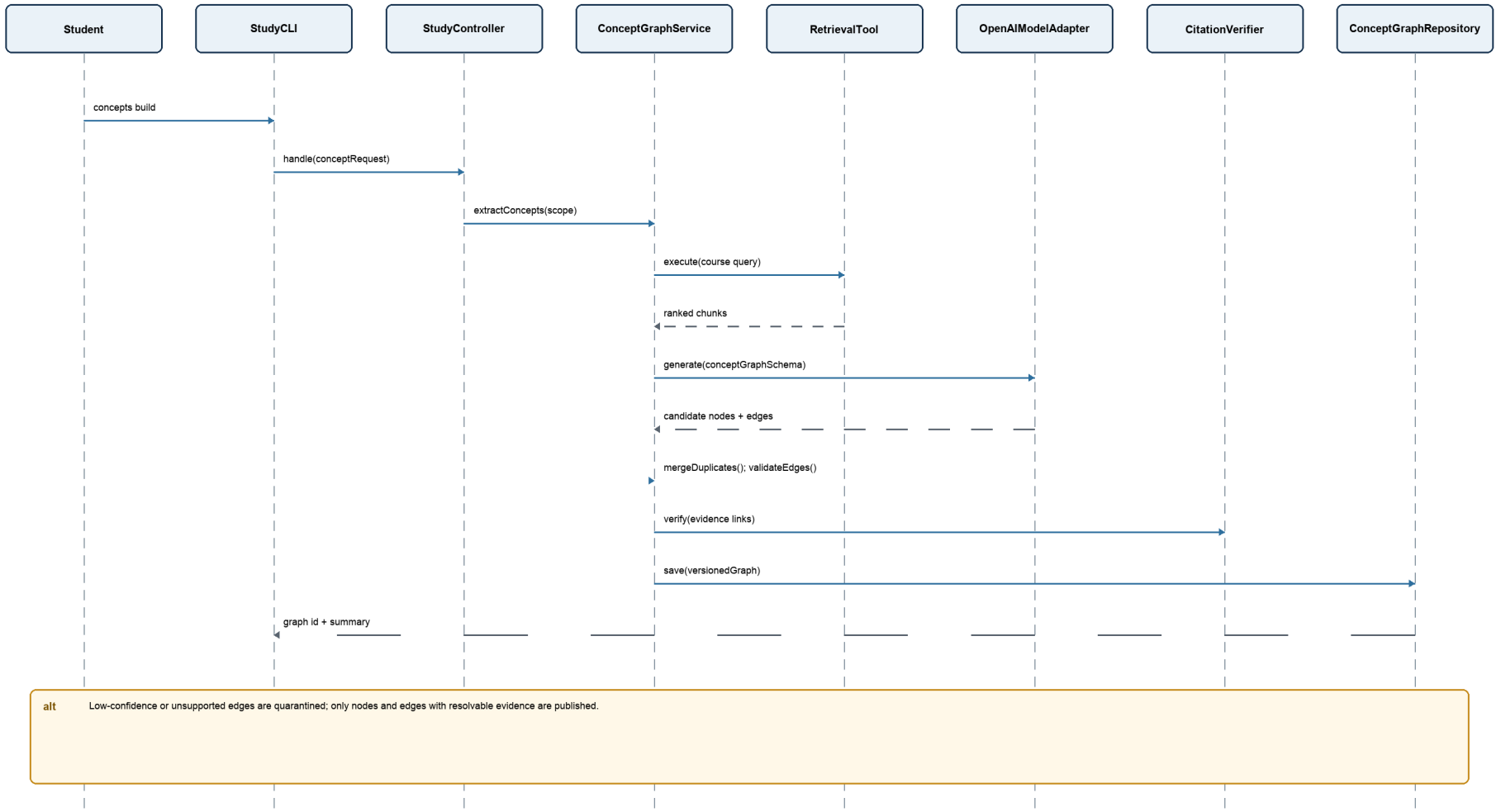


Figure 10. Build an evidence-linked concept map. Feature coverage: F03.

SD08 Schedule confirmed reminders

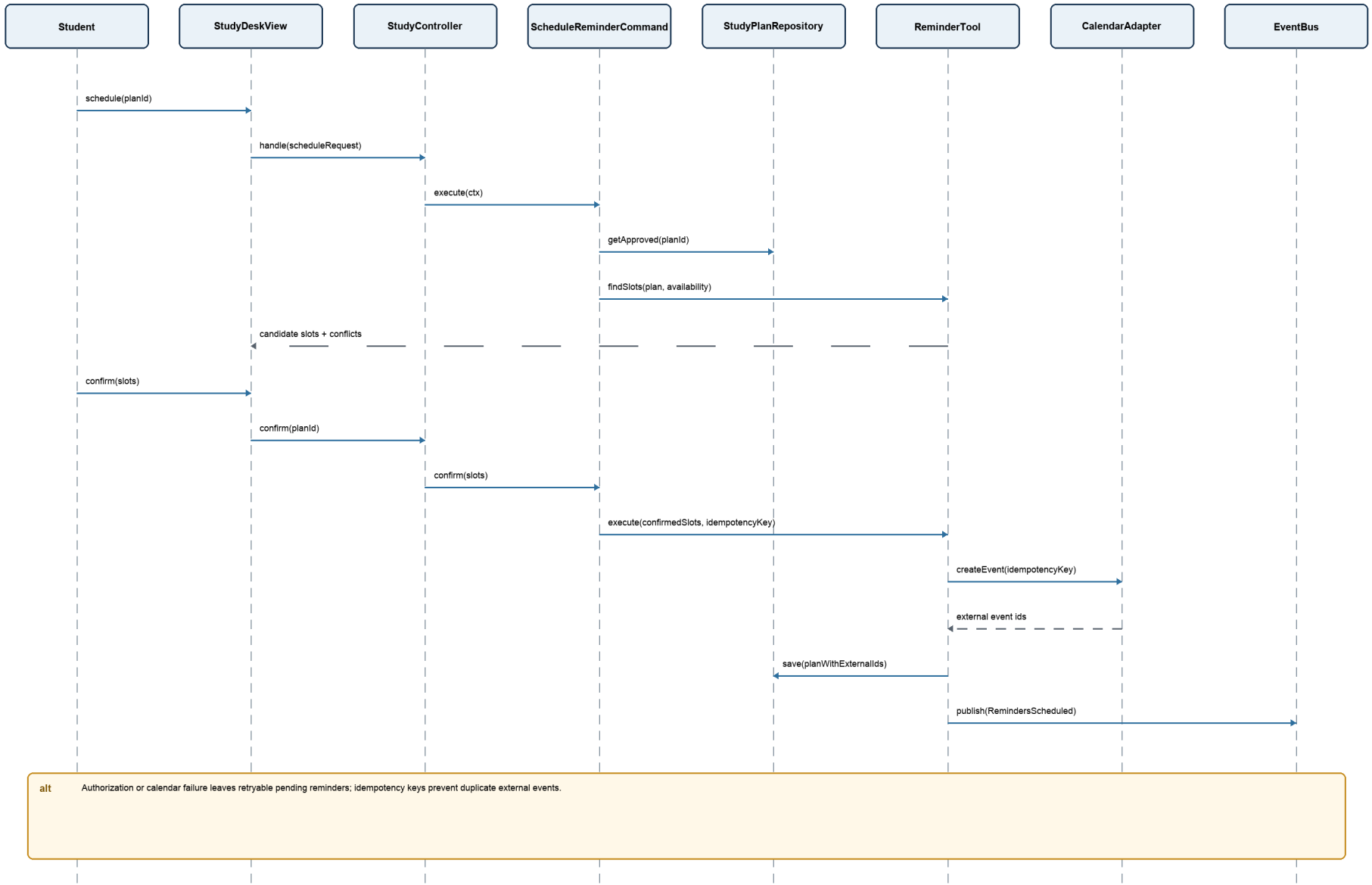


Figure 11. Schedule confirmed reminders. Feature coverage: F11.

SD09 Diagnose weak topics and adapt practice

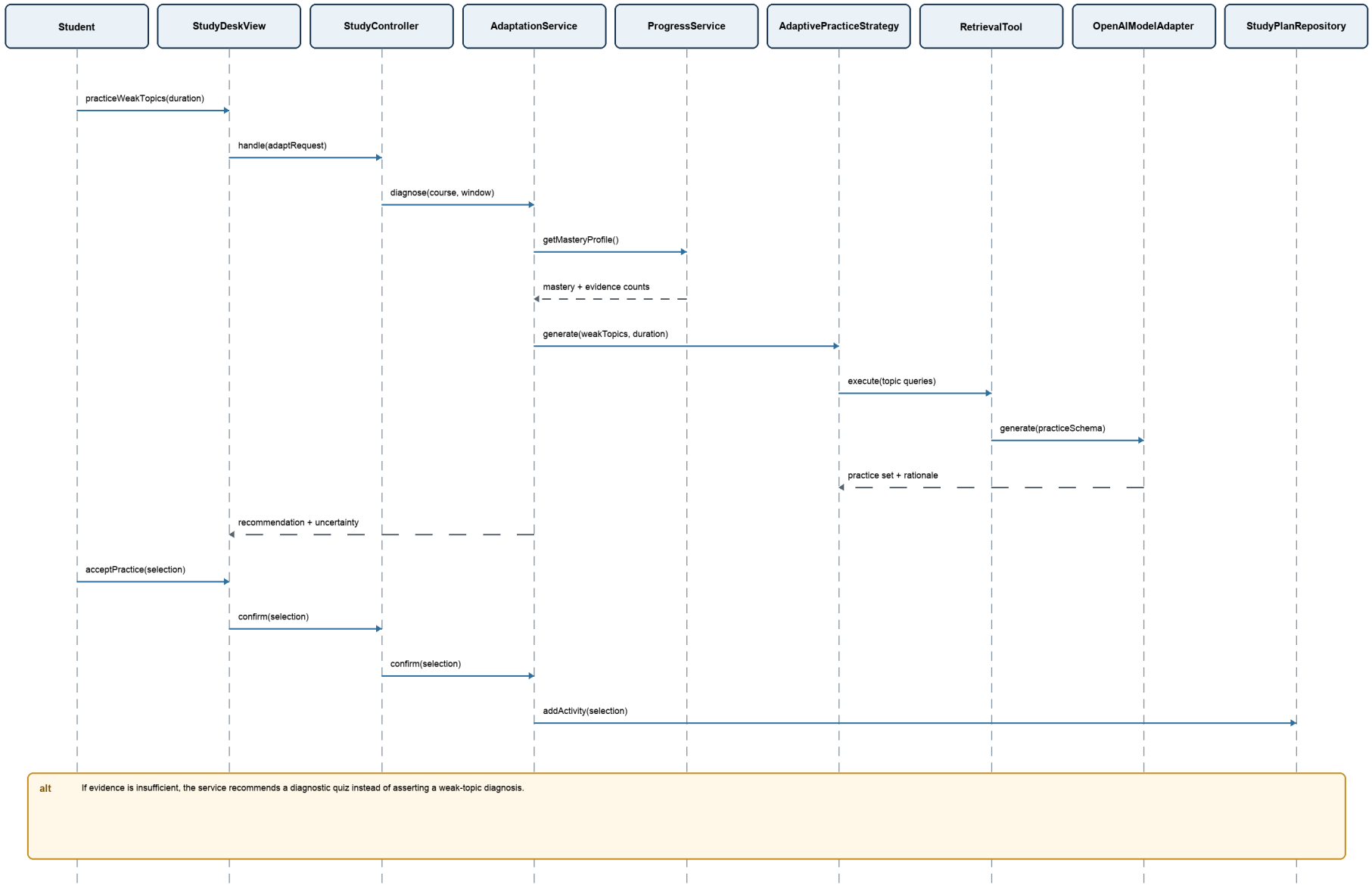


Figure 12. Diagnose weak topics and adapt practice. Feature coverage: F09.

SD10 View mastery analytics

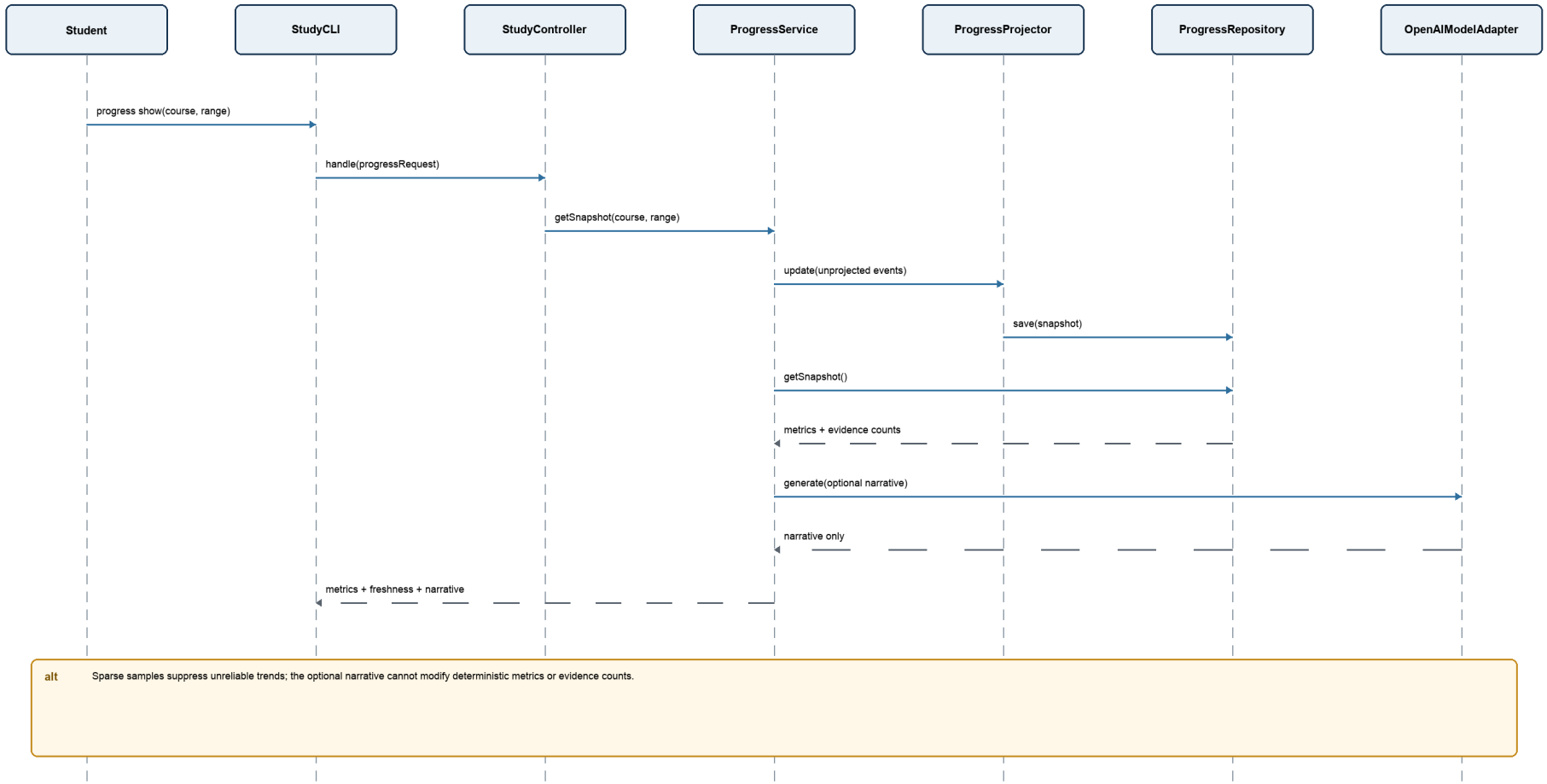


Figure 13. View mastery analytics. Feature coverage: F10.

SD11 Review flashcards and schedule next review

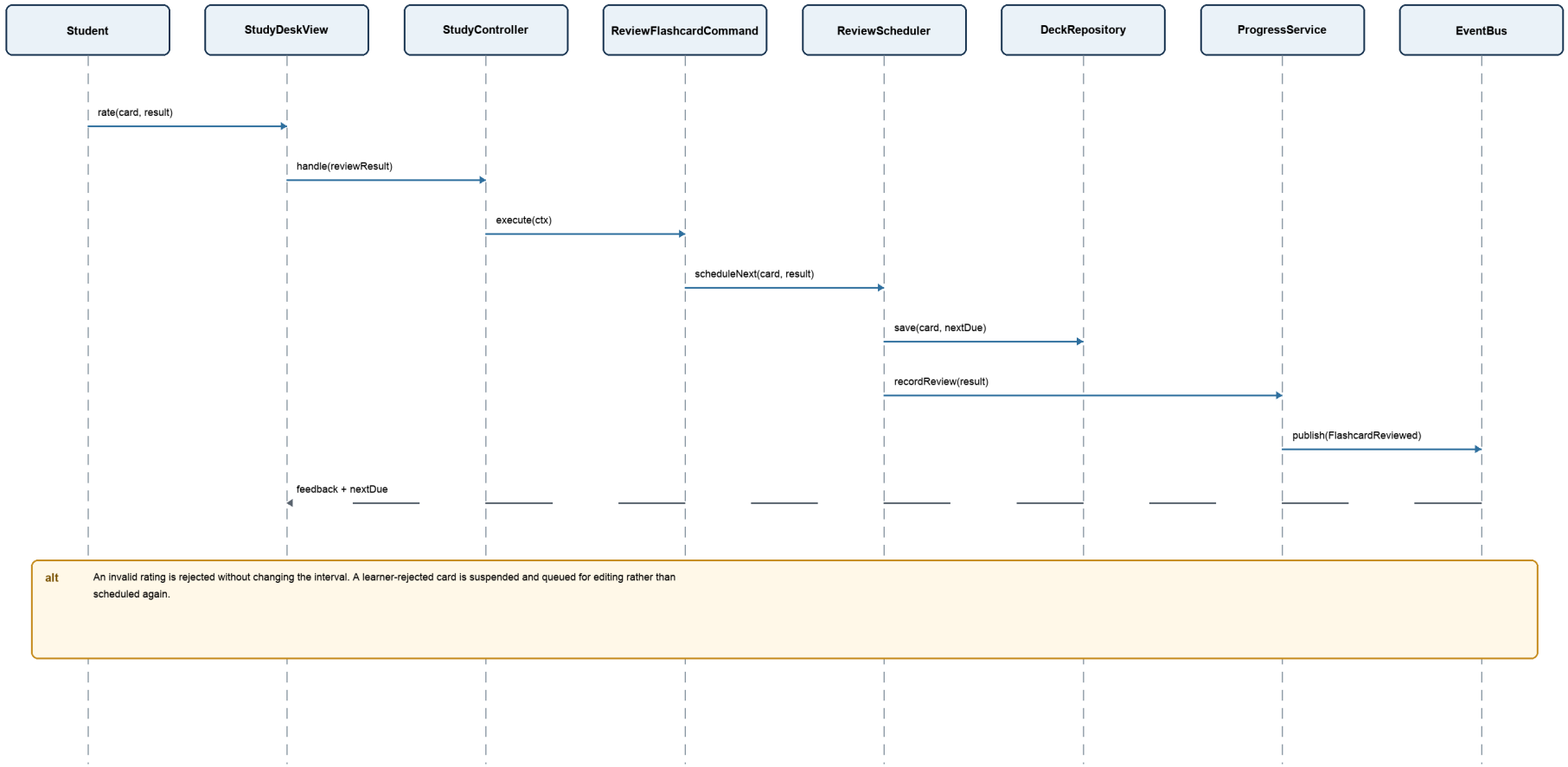


Figure 14. Review flashcards and schedule next review. Feature coverage: F06.

8 Feature-to-Design Traceability

Every feature is connected to a user-visible use case, a runtime diagram, concrete classes and methods, and patterns that solve its design problems. This table is the Stage 2 change-control baseline.

Feature	Description / type	Use case	Main classes	Key methods	Sequence	Patterns
F01	Material Ingestion and Indexing / Hybrid	UC01	MaterialRepository, DocumentIngestTool, EventBus	ImportMaterialCommand .execute(); DocumentIngestTool.execute(); Repository.save()	SD01	Command, Adapter, Observer
F02	Grounded Material Summaries / Hybrid	UC02	SummaryStrategy, RetrievalTool, ModelGateway, CitationVerifier, SummaryRepository	generate(); RetrievalTool.execute(); ModelGateway.generate(); verify(); save()	SD02	Strategy, Adapter, Facade
F03	Concept Extraction and Knowledge Map / Hybrid	UC03	ConceptGraphService, RetrievalTool, ModelGateway, CitationVerifier, ConceptGraphRepository	extractConcepts(); mergeDuplicates(); validateEdges(); verify(); save()	SD07	MVC, Adapter
F04	Grounded Question Answering / Hybrid	UC04	StudyAgentFacade, TaskPlanner, GroundedAnswerStrategy, RetrievalTool, ModelGateway, CitationVerifier, ConversationRepository	execute(); createPlan(); generate(); RetrievalTool.execute(); ModelGateway.generate(); verify(); save()	SD02	Facade, Strategy, Adapter
F05	Personalized Study Plan / Hybrid	UC05	StudyController, CreatePlanCommand, TaskPlanner, ProgressService, PlanValidator, StudyPlanRepository	handle(); confirm(); execute(); createPlan(); getMasteryProfile(); validate(); save()	SD03	MVC, Command, Adapter
F06	Flashcard Generation and Review / Hybrid	UC06	FlashcardStrategy, RetrievalTool, ReviewFlashcardCommand, ReviewScheduler, DeckRepository, ProgressService, EventBus	generate(); execute(); scheduleNext(); recordReview(); save(); publish()	SD04, SD11	Strategy, Command, Adapter, Observer

8 Feature-to-Design Traceability Continued

Feature	Description / type	Use case	Main classes	Key methods	Sequence	Patterns
F07	Quiz Generation / Hybrid	UC07	QuizStrategy, RetrievalTool, QuizValidator, QuizRepository	generate(); validateCoverage(); save()	SD04	Strategy, Adapter, Command
F08	Automatic Grading and Explanations / Hybrid	UC08	GradeAttemptCommand, AssessmentService, ModelGateway, AttemptRepository, EventBus	execute(); grade(); generate(); save(); publish()	SD05	Command, Adapter, Observer
F09	Weak-Topic Diagnosis and Adaptive Practice / Hybrid	UC09	StudyDeskView, StudyController, AdaptationService, AdaptivePracticeStrategy, RetrievalTool, ModelGateway, ProgressService, StudyPlanRepository	handle(); diagnose(); selectStrategy(); generate(); getMasteryProfile(); confirm(); addActivity()	SD09	MVC, Strategy, Adapter
F10	Progress and Mastery Analytics / Hybrid	UC10	ProgressService, ProgressProjector, ProgressRepository, ModelGateway, OpenAIModelAdapter	update(); getSnapshot(); generate()	SD10	Observer, MVC, Adapter
F11	Study Scheduling and Reminders / Hybrid	UC11	StudyDeskView, StudyController, ScheduleReminderCommand, StudyPlanRepository, ReminderTool, CalendarAdapter	handle(); confirm(); execute(); ReminderTool.execute(); createEvent(); save()	SD08	MVC, Command, Adapter
F12	Orchestrated Focused Study Session / Hybrid	UC12	RunSessionCommand, StudyAgentFacade, AgentOrchestrator, SessionState, ToolRegistry, SessionRepository, EventBus	execute(); run(); enter(); handle(); ToolRegistry.execute(); saveCheckpoint(); publish()	SD06	State, Facade, Command, Observer

9 Feature Realization Explanations

F01 Material Ingestion and Indexing

Related use case: UC01. Related sequence diagram: SD01. Patterns: Command, Adapter, Observer.

Classes and execution: StudyDeskView or StudyCLI sends an import request to StudyController, which creates ImportMaterialCommand. The command calls DocumentIngestTool.execute(), extracts and indexes chunks, requests proposed title and topic metadata through ModelGateway.generate(), persists Material through MaterialRepository.save(), and publishes MaterialIndexed. Per-file transaction boundaries preserve successful imports.

Important methods: ImportMaterialCommand.execute(); DocumentIngestTool.execute(); Repository.save()

F02 Grounded Material Summaries

Related use case: UC02. Related sequence diagram: SD02. Patterns: Strategy, Adapter, Facade.

Classes and execution: StudyController delegates the summary request through StudyAgentFacade. SummaryStrategy retrieves scoped chunks with RetrievalTool.execute(), builds a structured request, calls ModelGateway.generate(), parses the response, and requires CitationVerifier.verify() before SummaryRepository.save().

Important methods: generate(); RetrievalTool.execute(); ModelGateway.generate(); verify(); save()

F03 Concept Extraction and Knowledge Map

Related use case: UC03. Related sequence diagram: SD07. Patterns: MVC, Adapter.

Classes and execution: ConceptGraphService retrieves indexed chunks, invokes the concept extraction strategy, normalizes concept labels, merges duplicates, and validates typed edges. ConceptGraphRepository saves a versioned graph whose nodes and edges retain source chunk IDs.

Important methods: extractConcepts(); mergeDuplicates(); validateEdges(); verify(); save()

F04 Grounded Question Answering

Related use case: UC04. Related sequence diagram: SD02. Patterns: Facade, Strategy, Adapter.

Classes and execution: StudyAgentFacade.execute() asks IntentRouter to classify the question and TaskPlanner.createPlan() to create retrieval and answer steps. GroundedAnswerStrategy calls RetrievalTool, sends only ranked evidence to ModelGateway, verifies citations, saves the turn and evidence trail through ConversationRepository.save(), and returns the answer through the initiating boundary.

Important methods: execute(); createPlan(); generate(); RetrievalTool.execute(); ModelGateway.generate(); verify(); save()

F05 Personalized Study Plan

Related use case: UC05. Related sequence diagram: SD03. Patterns: MVC, Command, Adapter.

Classes and execution: CreatePlanCommand loads LearnerProfile and ProgressSnapshot, then TaskPlanner creates a candidate StudyPlan. Deterministic validators enforce availability, prerequisite order, deadline, and workload. The controller requests approval before StudyPlanRepository.save() replaces an active plan.

Important methods: handle(); confirm(); execute(); createPlan(); getMasteryProfile(); validate(); save()

F06 Flashcard Generation and Review

Related use case: UC06. Related sequence diagram: SD04, SD11. Patterns: Strategy, Command, Adapter, Observer.

Classes and execution: GenerateActivityCommand selects FlashcardStrategy. It retrieves evidence, generates typed cards, rejects unsupported or duplicate items, and saves a FlashcardDeck. During review, ReviewScheduler.scheduleNext() applies deterministic spaced-repetition rules and publishes a review event.

Important methods: generate(); execute(); scheduleNext(); recordReview(); save(); publish()

F07 Quiz Generation

Related use case: UC07. Related sequence diagram: SD04. Patterns: Strategy, Adapter, Command.

Classes and execution: GenerateActivityCommand selects QuizStrategy from the requested activity type. The strategy retrieves evidence, generates items and rubrics, and runs QuizValidator.validateCoverage(). Only answerable, unique, source-linked items reach QuizRepository.save().

Important methods: generate(); validateCoverage(); save()

F08 Automatic Grading and Explanations

Related use case: UC08. Related sequence diagram: SD05. Patterns: Command, Adapter, Observer.

Classes and execution: GradeAttemptCommand invokes AssessmentService.grade(). Objective items use deterministic graders; free-response items call ModelGateway.generate() with the rubric and evidence. The service calculates totals, flags low confidence, persists the finalized Attempt through AttemptRepository.save(), and publishes MasteryChanged only for accepted results.

Important methods: execute(); grade(); generate(); save(); publish()

F09 Weak-Topic Diagnosis and Adaptive Practice

Related use case: UC09. Related sequence diagram: SD09. Patterns: MVC, Strategy, Adapter.

Classes and execution: AdaptationService reads ProgressSnapshot, calculates evidence sufficiency, and identifies weak concepts and prerequisites. It selects AdaptivePracticeStrategy, generates a bounded activity, explains the choice, and waits for learner confirmation before StudyPlanRepository.addActivity() records it in the plan.

Important methods: handle(); diagnose(); selectStrategy(); generate(); getMasteryProfile(); confirm(); addActivity()

F10 Progress and Mastery Analytics

Related use case: UC10. Related sequence diagram: SD10. Patterns: Observer, MVC, Adapter.

Classes and execution: ProgressProjector observes AttemptRecorded, SessionAdvanced, and PlanSessionCompleted events and updates deterministic ProgressSnapshot projections. ProgressService returns metrics to GUI or CLI; an optional model request explains the already-calculated values but cannot change them.

Important methods: update(); getSnapshot(); generate()

F11 Study Scheduling and Reminders

Related use case: UC11. Related sequence diagram: SD08. Patterns: MVC, Command, Adapter.

Classes and execution: ScheduleReminderCommand loads an approved StudyPlan, computes candidate slots, and asks the learner to confirm through StudyController. ReminderTool executes through CalendarAdapter with an idempotency key, then StudyPlanRepository.save() stores the external event IDs. Failures leave a retryable pending record without duplicating events.

Important methods: handle(); confirm(); execute(); ReminderTool.execute(); createEvent(); save()

F12 Orchestrated Focused Study Session

Related use case: UC12. Related sequence diagram: SD06. Patterns: State, Facade, Command, Observer.

Classes and execution: RunSessionCommand calls StudyAgentFacade.execute(), and AgentOrchestrator runs the approved plan. StudySession delegates legal behavior to SessionState objects, ToolRegistry executes each activity, SessionRepository.saveCheckpoint() persists progress, and EventBus publishes UI updates. Tool errors enter a recoverable state; pause, skip, and stop remain learner-controlled.

Important methods: execute(); run(); enter(); handle(); ToolRegistry.execute(); saveCheckpoint(); publish()

10 Important Design Decisions and Quality Attributes

Decision	Consequence
Shared application core	GUI and CLI are adapters over the same commands and facade, preventing feature drift.
Grounded generation	Retrieved chunks and stable source locations are mandatory for summaries, answers, cards, quizzes, and explanations.
Bounded agency	The agent may choose read-only/local steps, but external writes and destructive replacements require confirmation.
Deterministic core	Dates, constraints, calculations, schemas, citations, permissions, idempotency, and state transitions remain conventional software.
Pinned model behind a port	A provider snapshot improves reproducibility, while ModelGateway prevents provider lock-in.
Event-driven projections	Progress and audit consumers react to domain events without coupling assessment or session services to dashboards.
Checkpointed state machine	Every completed activity is persisted before the next state, supporting safe pause/resume and failure recovery.
Explainable adaptation	Recommendations include evidence count, weak concepts, and constraints; the learner can override them.

10.1 Quality Attribute Scenarios

Reliability: When the model returns malformed structured output, the parser rejects it, performs one bounded retry, and returns a safe error without persisting partial domain state.

Traceability: Given any generated statement, the user can navigate to the material, page/section, and chunk used as evidence.

Testability: All deterministic services run against in-memory repositories and fake ports; model behavior is tested separately through contract and behavioral tests.

Modifiability: Adding a new learning activity requires a new strategy and command registration, not changes across both presentation layers.

Security/privacy: Only selected course chunks and necessary learner context are sent to the model; secrets remain in environment/configuration, never domain objects or logs.

Accessibility: Every major GUI capability has a CLI command, keyboard-accessible workflow, and text output.

Performance: Retrieval and deterministic validation occur before generation; expensive calls are cached by content hash, prompt version, model snapshot, and schema version.

10.2 Principal Risks and Mitigations

Risk	Mitigation
Unsupported or hallucinated content	Require scoped retrieval, structured citations, citation verification, and explicit evidence-gap results.
Model variability	Pin the model snapshot; version prompts and schemas; use contract tests and KUMA behavioral tests in Stage 3.
Prompt injection in uploaded material	Treat documents as data, delimit retrieved text, prevent it from changing system/tool policy, and allow only registered typed tools.
Incorrect free-text grading	Use explicit rubrics, confidence thresholds, learner review, and keep objective grading deterministic.
Calendar duplication	Use confirmation, idempotency keys, stored external IDs, and retryable pending status.
Over-adaptation from sparse data	Require minimum evidence, expose uncertainty, and offer diagnostics instead of confident diagnosis.
Scope growth	Stage 2 baseline excludes LMS integration, web search, collaborative roles, and automatic coursework submission.

11 Stage 2 Implementation Boundary

Stage 2 should implement a vertical slice before broadening feature coverage: shared request model, GUI and CLI boundaries, StudyController, CommandFactory, StudyAgentFacade, one repository adapter, OpenAIModelAdapter, ingestion, retrieval, grounded Q&A, and a checkpointed session. Subsequent increments add plan/schedule, practice/assessment, and progress/adaptation. Any material deviation from class, method, use-case, or pattern mappings in this report should be recorded as Original design -> Change -> Reason.

11.1 Suggested Package Structure

```
studyforge/
  presentation/gui   presentation/cli
  application/commands   application/facade
  agent/planning   agent/orchestration   agent/state
  domain/model   domain/services   domain/events
  ports/model   ports/tools   ports/repositories
  infrastructure/openai   infrastructure/storage   infrastructure/calendar
  tests/unit   tests/integration   tests/behavior
```

11.2 Verification Plan

- Unit-test parsers, validators, scheduling, mastery calculations, state transitions, idempotency, and repositories.
- Contract-test adapters against fake HTTP and file/calendar fixtures.
- Integration-test each command through both GUI presenter and CLI parser into the shared controller.
- Use golden source fixtures to verify that every citation resolves to the expected material location.
- Use KUMA in Stage 3 for tool selection, evidence use, instruction following, recovery, and multi-step completion.
- Diff Stage 2 behavior and class names against the traceability baseline; justify intentional changes.

References

1. EECS3311 Software Design. Course Project: Design, Build, and Test an AI Agent. Project-Instruction.pdf, Fall 2026.

2. EECS3311 Software Design. Course Project Stage-1 Instruction. stage1.pdf, Fall 2026.

3. OpenAI. GPT-4.1 mini model documentation. https://developers.openai.com/api/docs/models/gpt-4.1-mini (accessed October 9, 2026).

4. Gamma, E., Helm, R., Johnson, R., and Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

5. Object Management Group. OMG Unified Modeling Language specification, Version 2.5.1. https://www.omg.org/spec/UML/2.5.1/